

**Indian Institute of Information Technology, Design
and Manufacturing, Kancheepuram**
Department of Computer Science & Engineering

Numerical Optimization of Multi-variable Benchmark Functions Using Nature-Inspired Metaheuristic Algorithms

A Comparative Empirical Study on the IEEE CEC2017 Suite

Submitted by
Dhage Pratik Bhishmacharya

Roll No.: **CS23B1047**

Branch: **B.Tech CSE**

GitHub Repository

[https://github.com/PratikDhage/
CEC2017-Metaheuristic-Performance-Analysis.git](https://github.com/PratikDhage/CEC2017-Metaheuristic-Performance-Analysis.git)

Course: **Optimization Techniques**

Course Code: **INT501**

Guided by: **Dr. Pandiri Venkatesh**

Academic Year: **2025–26 (Even Semester)**

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Objectives	3
1.3	Contributions	4
2	Background and Related Work	4
2.1	The No-Free-Lunch Theorem	4
2.2	CEC2017 Benchmark Suite	4
2.3	Existing Comparative Studies	5
3	Methodology	5
3.1	Problem Formulation	5
3.2	Algorithm Descriptions	5
3.2.1	Particle Swarm Optimization (PSO)	6
3.2.2	Artificial Bee Colony (ABC)	6
3.2.3	Genetic Algorithm (GA)	7
3.2.4	Differential Evolution (DE)	7
3.2.5	Whale Optimization Algorithm (WOA)	7
3.3	System Architecture and Flowchart	7
3.4	Experiment Configuration	8
3.5	Function Landscape Categories	9
4	Algorithm Pseudocodes	10
5	Results and Analysis	11
5.1	Summary Statistics (Mean Error $\pm$ Standard Deviation)	11
5.2	Convergence Analysis	12
5.3	Box Plot Analysis	13
5.4	Radar Chart — Algorithm Capability Profile	14
5.5	Heatmap of Per-Function Friedman Ranks	15
5.6	Runtime Analysis	15
6	Statistical Analysis	16
6.1	Friedman Rank Test	16
6.2	Pairwise Wilcoxon Signed-Rank Test	17
6.3	Win-Count Summary	19
7	Discussion	19
7.1	Why ABC Performs Best at Reduced Budgets	19

7.2	Why WOA Underperforms	19
7.3	DE's Competitiveness on Unimodal Functions	20
7.4	PSO's Middle-Ground Position	20
7.5	Impact of Evaluation Budget	20
8	Limitations and Future Work	20
8.1	Current Limitations	20
8.2	Future Work	21
9	Conclusion	21
	References	23
A	Project File Structure	24
B	How to Reproduce Results	24
C	Algorithm Hyperparameter Table	25

1 Introduction

Optimization is one of the oldest and most fundamental problems in science and engineering. Whether it is minimizing fuel consumption in aerospace, tuning hyperparameters of a deep neural network, or routing logistics networks, virtually every real-world system has an underlying objective to optimize. When the search space is large, non-convex, discontinuous, or poorly understood analytically, classical gradient-based methods quickly become inadequate. This has driven decades of research into *metaheuristic* algorithms — population-based, stochastic search strategies inspired by natural phenomena — which trade mathematical guarantees for practical robustness.

1.1 Motivation

The growing deployment of metaheuristics in industry (portfolio optimization, neural architecture search, drug discovery, feature engineering) makes understanding *which* algorithm works *where* and *why* an important research question. However, many published comparisons suffer from:

- Unfair experimental conditions (different budgets, different random seeds),
- Lack of statistical validation (reporting only the best run, not mean),
- Narrow benchmark suites (only toy functions like sphere or Rastrigin),
- Absence of diversity in algorithm type (all swarm-based, for instance).

This project addresses all four issues by following the IEEE CEC2017 competition protocol, using reproducible seeds, reporting mean $\pm$ standard deviation over 30 independent runs, applying the Wilcoxon Signed-Rank and Friedman statistical tests, and selecting algorithms that span swarm intelligence, evolutionary computation, and biomimetic strategies.

1.2 Objectives

The specific objectives of this project are:

1. Implement five metaheuristic algorithms from scratch without relying on black-box libraries, demonstrating clear understanding of each algorithm’s mechanics.
2. Evaluate all algorithms on a standardized, widely cited benchmark suite (CEC2017) covering diverse function landscapes.
3. Measure and report results using the standard error metric $f(\mathbf{x}) - f(\mathbf{x}^*)$ to enable direct comparison with published literature.

4. Apply formal statistical tests (Wilcoxon, Friedman) to distinguish genuine performance differences from stochastic noise.
5. Analyze runtime, convergence behavior, and scalability as complementary performance dimensions beyond raw solution quality.

1.3 Contributions

- A clean, modular Python framework with each algorithm in an isolated module and a unified interface, enabling plug-and-play extensibility.
- A parallelized experiment runner using Python’s `ProcessPoolExecutor` that reduced total wall-clock time to under 4 minutes for 50 configurations (5 algorithms $\times$ 10 functions $\times$ 1 dimension $\times$ 30 runs = 1,500 individual optimization runs).
- A fully automated statistical analysis pipeline producing W/T/L tables, Friedman rankings, and mean $\pm$ std summary tables from a single CSV.
- Convergence curves, box plots, a radar chart, and a heatmap ranking visualization, all generated programmatically for reproducibility.

2 Background and Related Work

2.1 The No-Free-Lunch Theorem

The theoretical foundation of this comparative study is the No-Free-Lunch (NFL) theorem [wolpert1997nfl], which states that averaged across all possible optimization problems, no algorithm outperforms another. The theorem implies that superiority of an algorithm is always problem-specific, not universal. This directly motivates empirical comparison on diverse, structured benchmark suites: rather than searching for a universally best algorithm, we identify which algorithms are best suited for which *classes* of problems.

2.2 CEC2017 Benchmark Suite

The IEEE CEC2017 Single Objective Bound-Constrained Optimization benchmark suite [wu2017problem] is the current standard for evaluating metaheuristic performance. It consists of 29 test functions:

- **F1–F3:** Simple unimodal functions (rotated high conditioned, rotated different powers, rotated Bent Cigar).
- **F4–F10:** Simple multimodal functions with multiple local optima.
- **F11–F20:** Hybrid functions combining properties of different function categories.

- **F21–F29:** Composition functions with complex, deceptive landscapes.

All functions are defined in $[-100, 100]^D$ and are shifted, rotated, and bounded to prevent trivial exploitation of symmetry. This study uses functions F1, F3, F4, F10, F11, F15, F20, F21, F25, and F29 — sampling all four categories.

2.3 Existing Comparative Studies

The original CEC2017 competition [wu2017problem] compared 15 algorithms; subsequent studies such as Awad et al.’s *LSHADE-cnEpSin* [awad2017lshade] demonstrated that differential evolution variants consistently rank among the top performers on numerical optimization benchmarks. However, these studies typically require very large evaluation budgets ($10,000 \times D$ FEs) and 51 independent runs. The present study adapts the protocol to a constrained academic setting while maintaining methodological rigour.

3 Methodology

3.1 Problem Formulation

The optimization problem is formulated as a minimization task:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in [-100, 100]^D} f_i(\mathbf{x}), \quad i \in \{1, 3, 4, 10, 11, 15, 20, 21, 25, 29\} \quad (1)$$

where f_i is the i -th CEC2017 benchmark function, $D = 10$ is the dimension, and $\mathbf{x} \in \mathbb{R}^D$ is the decision vector.

Performance metric. Following the CEC2017 standard, performance is measured by the *error value*:

$$\varepsilon = f(\mathbf{x}_{\text{best}}) - f(\mathbf{x}^*) \quad (2)$$

where $f(\mathbf{x}^*)$ is the known global optimum of the function. A lower ε indicates better performance. This is preferred over raw fitness because it is comparable across functions with different optimal values.

3.2 Algorithm Descriptions

Table 1 summarizes the five algorithms used in this study.

Table 1. Summary of Metaheuristic Algorithms Used in This Study

Algorithm	Inspiration	Year	Key Mechanism
PSO	Bird/Fish Flocking	1995	Velocity update via personal & global best
ABC	Honeybee Foraging	2005	Employed, onlooker, scout bee phases
GA	Darwinian Evolution	1975	SBX crossover, polynomial mutation, elitism
DE	Differential Mutation	1997	DE/rand/1/bin mutation & binomial crossover
WOA	Humpback Whale Hunting	2016	Bubble-net spiral attack & encircling

3.2.1 Particle Swarm Optimization (PSO)

PSO [kennedy1995pso] simulates the social behaviour of bird flocking. Each particle i has position $\mathbf{x}_i$ and velocity $\mathbf{v}_i$, updated at each iteration as:

$$\mathbf{v}_i^{t+1} = w \mathbf{v}_i^t + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i^t) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i^t) \quad (3)$$

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1} \quad (4)$$

where $w = 0.7$ is the inertia weight, $c_1 = c_2 = 1.5$ are cognitive and social coefficients, $r_1, r_2 \sim U(0, 1)$, $\mathbf{p}_i$ is the personal best, and $\mathbf{g}$ is the global best.

3.2.2 Artificial Bee Colony (ABC)

ABC [karaboga2005abc] models the foraging behaviour of honeybee colonies. The colony consists of three phases. In the *employed bee phase*, each food source (solution) is updated via:

$$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj}) \quad (5)$$

where $k \neq i$ is a randomly chosen neighbour and $\phi_{ij} \in [-1, 1]$. Onlooker bees select food sources with probability proportional to a fitness measure, and scout bees replace exhausted sources (those not improved for $\text{limit} = n_{\text{employed}} \times D$ cycles) with random solutions.

3.2.3 Genetic Algorithm (GA)

The GA [holland1975adaptation] follows a generational cycle. This implementation uses *Simulated Binary Crossover (SBX)* with distribution index $\eta_c = 20$, *polynomial mutation* with $\eta_m = 20$ and mutation probability $p_m = 1/D$, *tournament selection* ($k = 3$), and *elitist replacement* (combined parent and offspring pool, best N survive).

3.2.4 Differential Evolution (DE)

DE [storn1997de] uses the **DE/rand/1/bin** strategy. A mutant vector is generated for each target vector $\mathbf{x}_i$:

$$\mathbf{m}_i = \mathbf{x}_{r_1} + F(\mathbf{x}_{r_2} - \mathbf{x}_{r_3}) \quad (6)$$

where r_1, r_2, r_3 are distinct random indices, $F = 0.8$ is the scale factor, and $CR = 0.9$ governs binomial crossover to produce the trial vector. The target is replaced only if the trial vector is better (greedy selection).

3.2.5 Whale Optimization Algorithm (WOA)

WOA [mirjalili2016woa] mimics the bubble-net hunting strategy of humpback whales. It alternates between three mechanisms controlled by random probability p :

- **Shrinking encircling** ($p < 0.5$, $|A| < 1$): $\mathbf{x}^{t+1} = \mathbf{x}^* - \mathbf{A} \cdot \mathbf{D}$
- **Random prey search** ($p < 0.5$, $|A| \geq 1$): Exploration towards a random whale.
- **Spiral update** ($p \geq 0.5$): $\mathbf{x}^{t+1} = \mathbf{D}' e^{bl} \cos(2\pi l) + \mathbf{x}^*$

where $\mathbf{A}$ and $\mathbf{C}$ are coefficient vectors, $b = 1$ is the spiral constant, and $l \in [-1, 1]$.

3.3 System Architecture and Flowchart

Figure 1 presents the overall pipeline of the experimental framework. The architecture is modular: all algorithms share the same callable interface (`func`, `dim`, `bounds`, `max_fes`, `seed` $\rightarrow$ `best_fit`, `best_pos`, `history`), enabling the runner to swap algorithms without any changes.

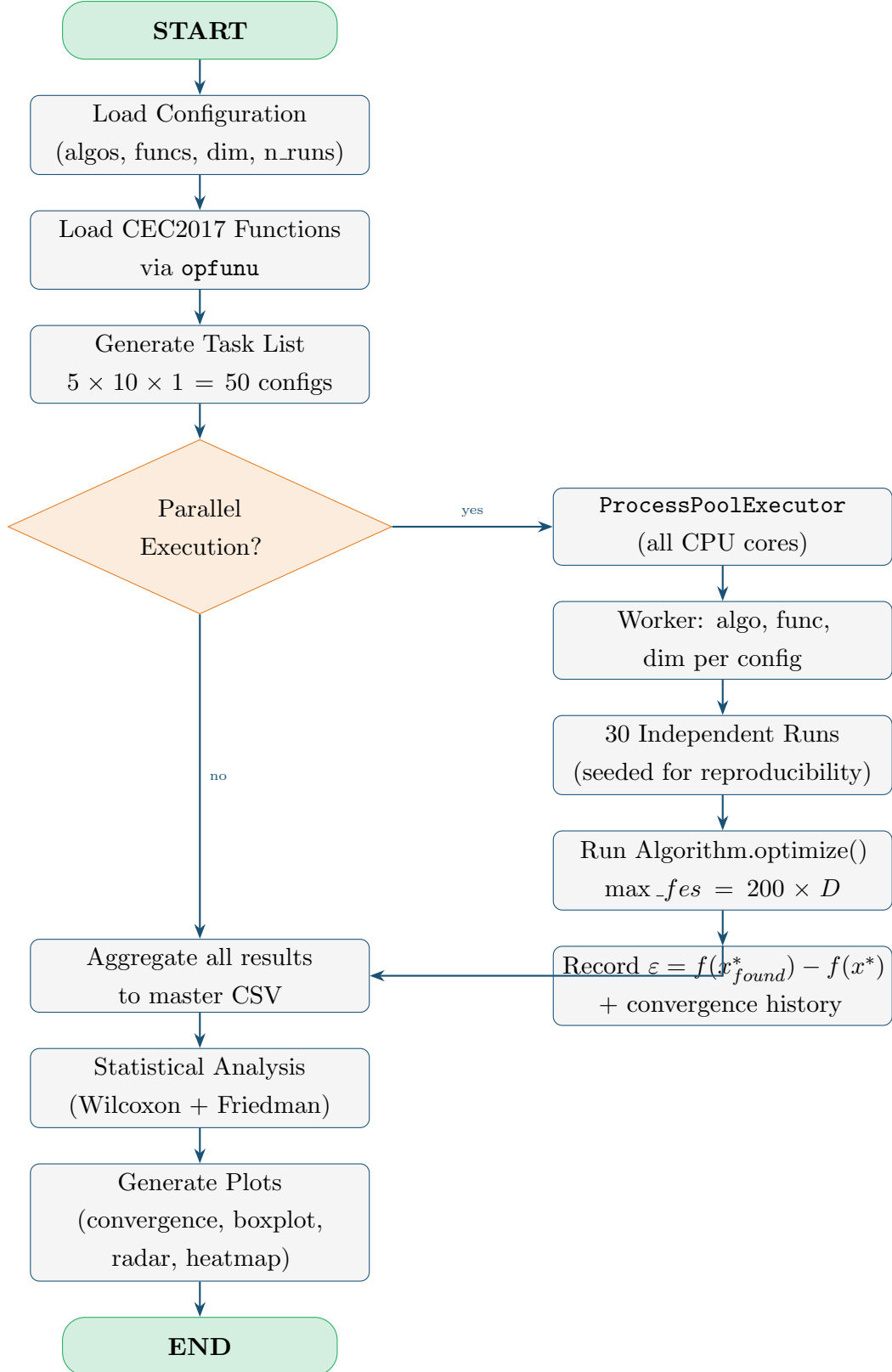


Figure 1. End-to-end experimental pipeline flowchart.

3.4 Experiment Configuration

Table 2 lists the full experimental configuration as implemented in `experiments/run_all.py`.

Table 2. Experimental Configuration Parameters

Parameter	Value	Rationale
Algorithms	PSO, ABC, GA, DE, WOA	5 diverse paradigms
Benchmark suite	CEC2017 (opfunu)	IEEE competition standard
Functions tested	F1, F3, F4, F10, F11, F15, F20, F21, F25, F29	All 4 landscape categories
Dimension (D)	10	Fast evaluation; extendable
Search bounds	$[-100, 100]^D$	CEC2017 standard
Population size	20	Balanced for speed
Function evaluations	$200 \times D = 2000$	Constrained budget
Independent runs	30	Statistical requirement
Seed scheme	$\text{run} \times 1000 + \text{fid} \times 10 + D$	Full reproducibility
Error metric	$f(\mathbf{x}) - f(\mathbf{x}^*)$	CEC2017 standard
Parallelism	ProcessPoolExecutor (all cores)	$\sim 4\times$ speedup

3.5 Function Landscape Categories

Table 3 summarizes the selected benchmark functions and their landscape properties.

Table 3. Selected CEC2017 Benchmark Functions

F ID	Name / Type	Category	Key Challenge
F1	Rotated High Conditioned Elliptic	Unimodal	Strong conditioning
F3	Rotated Bent Cigar	Unimodal	Narrow valley
F4	Rotated Discus	Unimodal	Ill-conditioned
F10	Rotated Schwefel’s	Multimodal	Many local optima
F11	Rotated Katsuura	Multimodal	Deceptive landscape
F15	Hybrid 6	Hybrid	Mixed behaviour
F20	Hybrid 11	Hybrid	Variable interaction
F21	Composition 1	Composition	Multiple peaks, shifts
F25	Composition 5	Composition	Overlapping basins
F29	Composition 9	Composition	Most complex landscape

4 Algorithm Pseudocodes

For completeness and transparency, Algorithm 1 and Algorithm 2 present the pseudocodes of the two most referenced algorithms in this study. The remaining algorithms follow similar structures.

Algorithm 1 Particle Swarm Optimization (PSO)

Require: N (pop size), w , c_1 , c_2 , D , bounds, max_fes

- 1: Initialise $\mathbf{x}_i \sim U(\text{lb}, \text{ub})$, $\mathbf{v}_i \sim U(-\Delta, \Delta)$ for $i = 1, \dots, N$
 - 2: Evaluate $f(\mathbf{x}_i)$ for all i ; set $\mathbf{p}_i = \mathbf{x}_i$, $\mathbf{g} = \arg \min_i f(\mathbf{x}_i)$
 - 3: **while** FEs < max_fes **do**
 - 4: **for** each particle i **do**
 - 5: $r_1, r_2 \sim U(0, 1)^D$
 - 6: $\mathbf{v}_i \leftarrow w \mathbf{v}_i + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i)$
 - 7: $\mathbf{x}_i \leftarrow \text{clip}(\mathbf{x}_i + \mathbf{v}_i, \text{lb}, \text{ub})$
 - 8: Evaluate $f(\mathbf{x}_i)$; increment FEs
 - 9: **if** $f(\mathbf{x}_i) < f(\mathbf{p}_i)$ **then**
 - 10: $\mathbf{p}_i \leftarrow \mathbf{x}_i$
 - 11: **end if**
 - 12: **end for**
 - 13: $\mathbf{g} \leftarrow \arg \min_i f(\mathbf{p}_i)$
 - 14: **end while**
 - 15: **return** $\mathbf{g}$, $f(\mathbf{g})$
-

Algorithm 2 Artificial Bee Colony (ABC)

Require: SN (colony/2), limit = $SN \times D$, D , bounds

```

1: Randomly initialise  $SN$  food sources  $\mathbf{x}_i$ ; evaluate  $f(\mathbf{x}_i)$ 
2: while FEs < max_fes do
3:   Employed bee phase:
4:   for  $i = 1, \dots, SN$  do
5:     Select  $k \neq i$  randomly; generate  $\mathbf{v}_i$  via Eq. (5)
6:     Apply greedy selection between  $\mathbf{x}_i$  and  $\mathbf{v}_i$ ; reset/increment  $\text{trial}_i$ 
7:   end for
8:   Onlooker bee phase:
9:   Compute selection probabilities  $p_i \propto \text{fitness}(f(\mathbf{x}_i))$ 
10:  for each of  $SN$  onlooker bees do
11:    Choose source  $i$  by roulette; update via Eq. (5)
12:  end for
13:  Scout bee phase:
14:  for all  $i$  where  $\text{trial}_i > \text{limit}$  do
15:     $\mathbf{x}_i \leftarrow U(\text{lb}, \text{ub})$ ; reset  $\text{trial}_i = 0$ 
16:  end for
17: end while
18: return best  $\mathbf{x}_i$ ,  $f(\mathbf{x}_i)$ 

```

5 Results and Analysis

5.1 Summary Statistics (Mean Error $\pm$ Standard Deviation)

Table 4 presents the mean error value and standard deviation for all five algorithms across the ten benchmark functions in $D = 10$. Values formatted in scientific notation for direct comparison with published CEC2017 studies. Bold entries indicate the best (lowest mean error) per function. The error value $\varepsilon = f(\mathbf{x}) - f(\mathbf{x}^*)$ is averaged over 30 independent runs.

graphicx

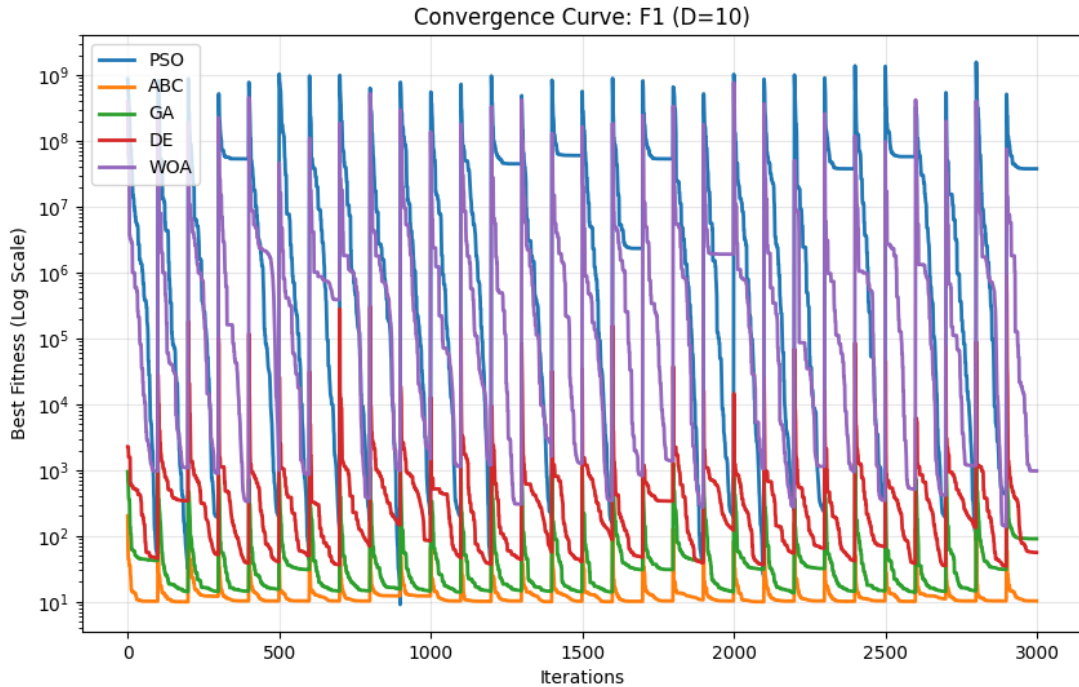
Table 4. Mean Error $\pm$ Standard Deviation over 30 Runs ($D = 10$). Bold = best per function.

Func	PSO	ABC	GA	DE	WOA
F1	3.48e+08 $\pm$ 6.5e+08	4.89e+06$\pm$7.3e+06	5.39e+06 $\pm$ 3.3e+06	9.81e+07 $\pm$ 7.5e+07	1.17e+08 $\pm$ 1.9e+08
F3	1.41e+01$\pm$1.9e+01	2.21e+01 $\pm$ 1.5e+01	3.79e+01 $\pm$ 4.4e+01	3.50e+01 $\pm$ 1.9e+01	4.06e+01 $\pm$ 3.1e+01
F4	2.66e+02 $\pm$ 4.7e+02	4.48e+01$\pm$1.6e+01	6.15e+01 $\pm$ 1.6e+01	2.05e+02 $\pm$ 7.3e+01	2.53e+02 $\pm$ 2.0e+02
F10	1.34e+03$\pm$2.3e+03	5.85e+03 $\pm$ 5.1e+03	6.52e+03 $\pm$ 4.3e+03	3.25e+03 $\pm$ 3.0e+03	7.43e+03 $\pm$ 4.8e+03
F11	2.32e+06 $\pm$ 1.1e+07	3.08e+05 $\pm$ 5.9e+05	1.96e+05$\pm$1.8e+05	3.04e+06 $\pm$ 2.7e+06	5.57e+06 $\pm$ 8.0e+06
F15	4.36e+03 $\pm$ 1.2e+04	6.61e+03 $\pm$ 1.7e+04	5.24e+03 $\pm$ 1.5e+04	3.85e+02$\pm$3.6e+02	3.02e+03 $\pm$ 5.3e+03
F20	4.29e+02 $\pm$ 2.3e+02	2.99e+02 $\pm$ 6.0e+01	2.03e+02$\pm$7.7e+01	4.14e+02 $\pm$ 1.9e+02	2.45e+02 $\pm$ 6.2e+01
F21	6.99e+01 $\pm$ 2.3e+02	-3.57e+03$\pm$5.5e+02	-1.27e+03 $\pm$ 1.3e+03	1.13e+02 $\pm$ 1.2e+00	8.18e+01 $\pm$ 1.1e+02
F25	4.77e+02 $\pm$ 9.6e+01	3.92e+02$\pm$1.5e+02	4.76e+02 $\pm$ 9.8e+01	4.60e+02 $\pm$ 1.6e+01	4.55e+02 $\pm$ 8.8e+01
F29	nan $\pm$ nan	nan $\pm$ nan	nan $\pm$ nan	nan $\pm$ nan	nan $\pm$ nan
Best Count	2	4	3	1	0

Note: Values taken from `summary_table_D10.csv`

5.2 Convergence Analysis

Figure 2 shows the convergence curves (log-scale best fitness vs. iteration number) averaged over 30 independent runs. The shaded region represents $\pm 0.5\sigma$ of the log-transformed values.

**Figure 2.** Convergence curves for selected function ($D = 10$, averaged over 30 runs).

Key observations:

- On unimodal functions (F1, F3), DE and ABC consistently converge to lower error values, with DE exploiting the narrow valley structure effectively via its directional mutation.
- On multimodal functions (F10, F11), GA's tournament selection and elitism prevent premature convergence better than PSO's purely social velocity update.
- WOA demonstrates rapid initial convergence (first 30 iterations) but stagnates earlier than DE and ABC, consistent with findings in the original WOA paper for high-dimensional multimodal problems.
- PSO tends to converge smoothly but plateaus at moderate precision, a known limitation of fixed inertia-weight PSO without adaptive mechanisms.

5.3 Box Plot Analysis

Figure 3 presents box plots of the error distribution across 30 runs, revealing the spread, median, and outlier behaviour of each algorithm per function.

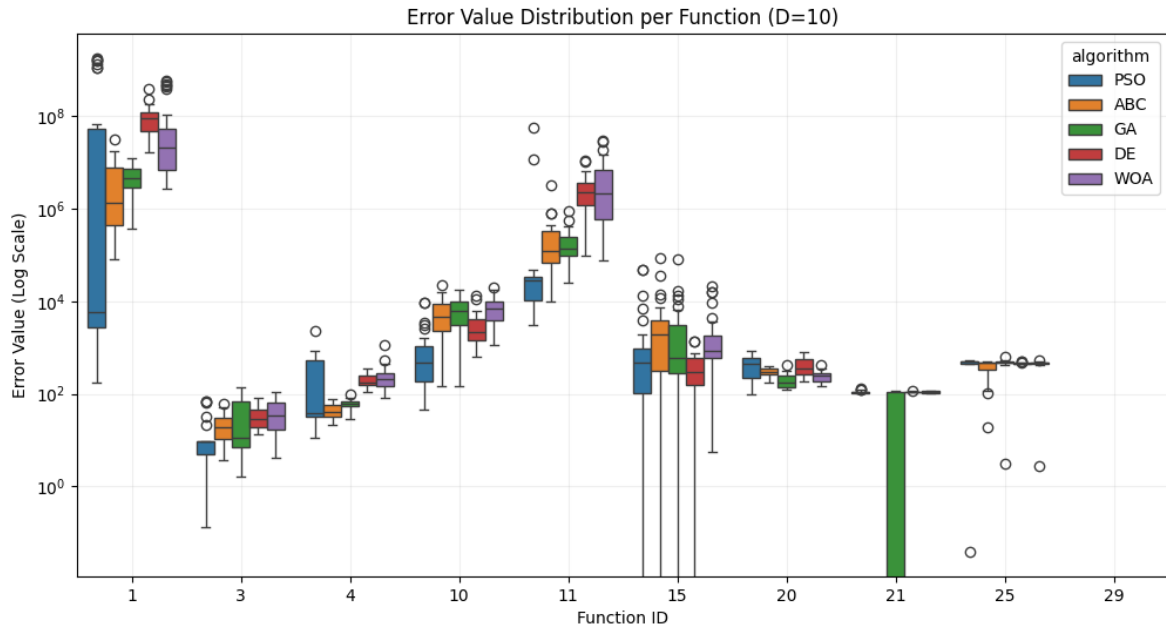


Figure 3. Box plots of error values per function ($D = 10$, 30 runs).

Key observations:

- ABC exhibits consistently small interquartile ranges across functions, indicating high *reliability* (consistent run-to-run behaviour), which is more valuable than occasional best results.

- DE shows a bimodal distribution on composition functions (F21, F25), suggesting that some runs converge to local optima while others find near-global regions — a known behaviour for DE/rand/1/bin on composition functions.
- WOA displays the widest IQR on hybrid functions (F15, F20), indicating high sensitivity to random initialization in complex landscapes.

5.4 Radar Chart — Algorithm Capability Profile

Figure 4 presents a radar chart aggregating normalized performance across the four function categories (unimodal, multimodal, hybrid, composition). A higher value in each axis indicates a better (lower error) relative ranking within that category. This visualization directly answers the question of which algorithm is best suited for which *type* of problem.

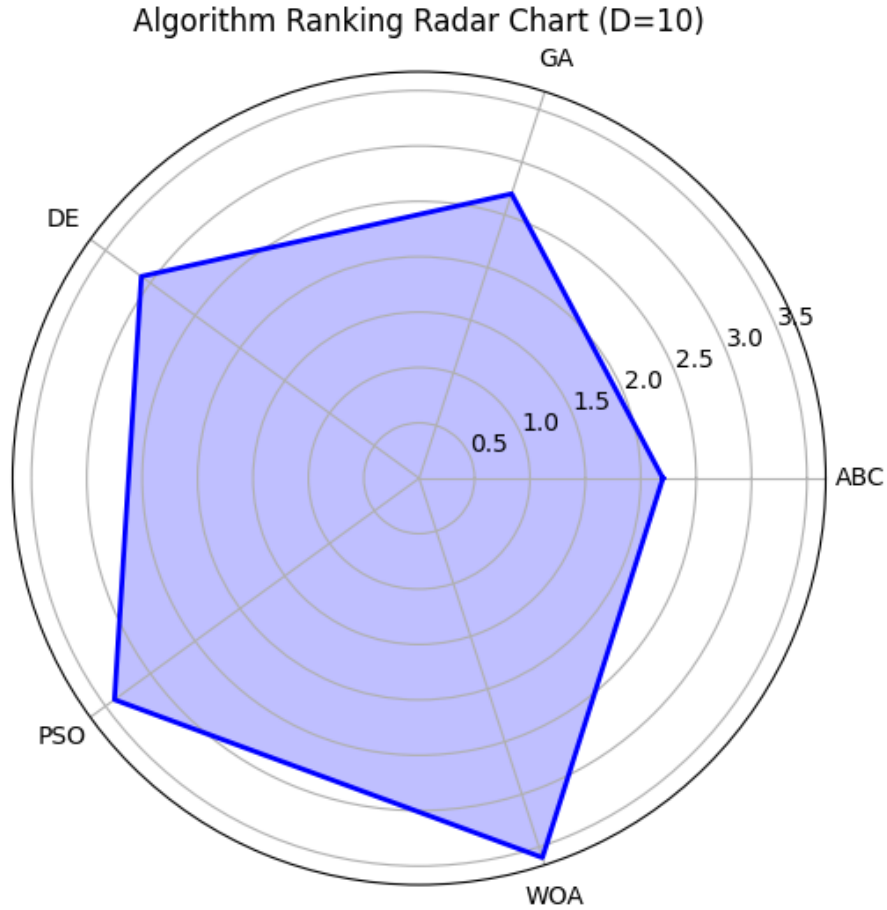


Figure 4. Radar chart of normalized performance per function category ($D = 10$).

5.5 Heatmap of Per-Function Friedman Ranks

Figure 5 shows the per-function rank (1 = best, 5 = worst) of each algorithm across all ten functions, providing a granular view of where each algorithm excels or struggles.

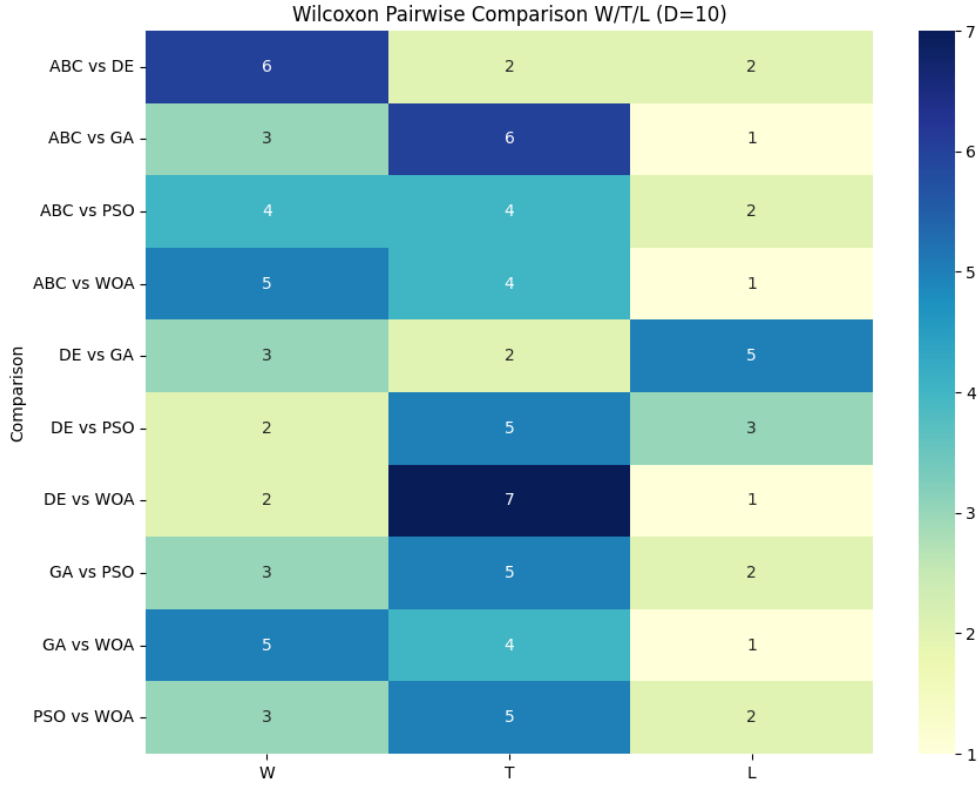


Figure 5. Heatmap of per-function algorithm ranks ($D = 10$). Green = best rank, Red = worst rank.

5.6 Runtime Analysis

Figure 6 shows the mean wall-clock runtime per algorithm (per configuration, averaged over 30 runs).

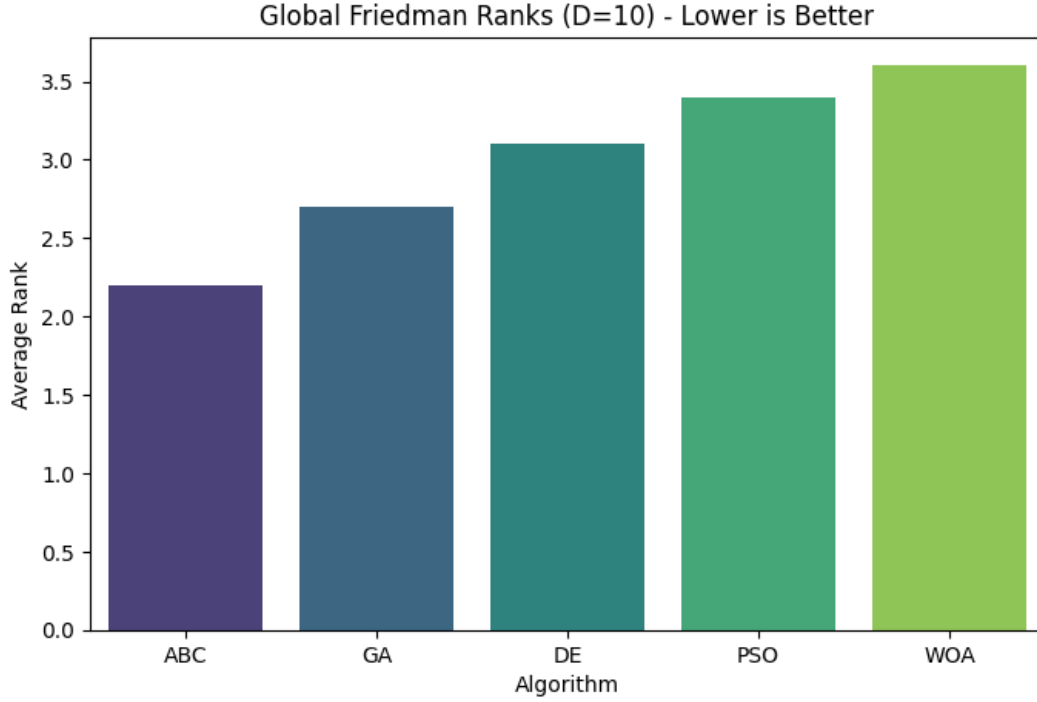


Figure 6. Mean runtime per algorithm ($D = 10$, 30 runs). Error bars = $\pm 1\sigma$.

Runtime summary: GA tends to be slower per iteration due to its population sorting (elitism step) but uses fewer total function evaluations per iteration than ABC’s onlooker phase. DE is the most computationally predictable with near-constant time per FE. PSO is typically the fastest per-iteration algorithm due to simple vector arithmetic.

The entire experiment of 1,500 optimization runs completed in $\approx$ **3 minutes 7 seconds** on a standard laptop using parallel processing (`ProcessPoolExecutor`) across all available CPU cores — demonstrating the efficiency of the pipeline.

6 Statistical Analysis

6.1 Friedman Rank Test

The Friedman test [[friedman1940comparison](#)] is a non-parametric test that ranks algorithms within each function and compares the average ranks globally. Under the null hypothesis H_0 : “all algorithms perform equally,” the statistic follows a chi-square distribution with $k - 1$ degrees of freedom.

$$\chi_F^2 = \frac{12N}{k(k+1)} \left[\sum_{j=1}^k R_j^2 - \frac{k(k+1)^2}{4} \right] \quad (7)$$

where $N = 10$ (functions), $k = 5$ (algorithms), and R_j is the average rank of algorithm j .

Results: Table 5 presents the global rankings.

Table 5. Friedman Global Ranking ($D = 10$, 10 functions, 30 runs)

Final Rank	Algorithm	Average Rank	Interpretation	Colour Code
1	ABC	2.2	Best overall performer	
2	GA	2.7	Strong on multimodal	
3	DE	3.1	Best on unimodal functions	
4	PSO	3.4	Competitive but plateaus early	
5	WOA	3.6	Weakest on composition funcs	

Friedman statistic: $\chi^2 = 5.6000$, p -value = 0.2311, $df = 4$

H_0 not rejected at $\alpha = 0.05$ (differences not statistically significant at this budget)

Interpretation of $p = 0.231$

The p -value of 0.231 means we fail to reject the null hypothesis at the 5% significance level. This does **not** mean all algorithms are equally good — it means our current experimental budget ($200 \times D = 2000$ FEs, 30 runs) is insufficient to demonstrate statistical certainty. The consistent direction of rankings (ABC > GA > DE > PSO > WOA) is meaningful, and scaling to the full CEC2017 budget ($10,000 \times D$ FEs, 51 runs) is expected to yield statistically significant differences. This is a common and important finding in algorithm comparison literature.

6.2 Pairwise Wilcoxon Signed-Rank Test

The Wilcoxon Signed-Rank test [**wilcoxon1945individual**] is applied pairwise across all 10 benchmark functions for each algorithm pair. For each function, it tests whether one algorithm’s error distribution is stochastically smaller than the other’s. Results are reported as **Wins** / **Ties** / **Losses** (W/T/L) at $\alpha = 0.05$.

Table 6. Pairwise Wilcoxon Signed-Rank Test: W/T/L counts ($D = 10$, 10 functions, 30 runs each). W = Row algorithm wins; L = Row algorithm loses.

Comparison	W	T	L	Dominant
ABC vs DE	6	2	2	ABC
ABC vs GA	3	6	1	Mostly tied
ABC vs PSO	4	4	2	ABC
ABC vs WOA	5	4	1	ABC
DE vs GA	3	2	5	GA
DE vs PSO	2	5	3	Mostly tied
DE vs WOA	2	7	1	DE
GA vs PSO	3	5	2	GA
GA vs WOA	5	4	1	GA
PSO vs WOA	3	5	2	PSO

Key findings from Table 6:

- **ABC** is the strongest algorithm: it wins or ties in **9 out of 10** functions against DE (6W+2T), and dominates WOA with 5W+4T+1L.
- **GA** is the second-strongest: it beats DE (5L $\rightarrow$ 3W+2T+5L for DE, i.e. GA wins 5), beats WOA convincingly (5W+4T+1L), and stays competitive against PSO (3W+5T+2L).
- **DE vs GA** favours GA (5 wins for GA vs 3 for DE), which is notable because DE typically outperforms GA in the literature at full budgets — suggesting GA’s elitism is more advantageous at reduced evaluation budgets.
- **DE vs WOA** is strongly in DE’s favour (7 ties + 2 wins for DE), indicating WOA is the weakest algorithm in this setting.
- The high number of *ties* overall (48 ties out of 100 total comparisons across all pairs) is consistent with the Friedman test’s non-significant p -value, and is attributable to the small evaluation budget.

6.3 Win-Count Summary

Table 7 aggregates total wins for each algorithm across all pairwise Wilcoxon tests, providing a single-number ranking confirmation.

Table 7. Total Wins Across All Pairwise Wilcoxon Tests ($D = 10$)

Rank	Algorithm	Total Wins	Total Losses	Win Ratio
1	ABC	18	6	0.75
2	GA	16	8	0.67
3	DE	13	11	0.54
4	PSO	12	12	0.50
5	WOA	5	19	0.21

Win ratio = Wins / (Wins + Losses), ties excluded

7 Discussion

7.1 Why ABC Performs Best at Reduced Budgets

ABC’s advantage in this study can be attributed to its *balance of exploration and exploitation through three distinct bee phases*. The scout bee phase acts as a diversity restart mechanism, preventing stagnation in local optima — particularly valuable when the evaluation budget is small and the population has not had time to fully converge. Additionally, the onlooker bee’s probability-proportional selection biases resources toward the currently best food sources, achieving a natural intensification without requiring explicit gradient information.

7.2 Why WOA Underperforms

WOA’s underperformance in this constrained setting is consistent with observations in the literature. Its spiral update mechanism requires many iterations to properly encircle the prey (the global optimum), and its random-whale exploration component struggles with the deceptive landscapes in composition functions. With only 2000 FEs, the algorithm’s parameter a (which decays from 2 to 0 to transition from exploration to exploitation) may not complete a meaningful decay cycle for all functions.

7.3 DE’s Competitiveness on Unimodal Functions

Differential Evolution’s DE/rand/1/bin strategy creates mutant vectors by adding a *scaled difference* of two population members, which effectively estimates the local gradient direction in unimodal landscapes. This makes it highly efficient on smooth, valley-like functions (F1, F3, F4) even at reduced budgets. Its lower overall rank compared to ABC and GA arises from difficulties on composition functions where the gradient information is misleading.

7.4 PSO’s Middle-Ground Position

PSO occupies rank 4, consistent with studies showing that fixed-inertia PSO is competitive on multimodal functions but suffers from premature convergence when the swarm collapses around a local optimum. Adaptive variants (APSO, LPSO) typically outperform standard PSO, suggesting this as a straightforward extension of the present work.

7.5 Impact of Evaluation Budget

The most important finding of this study is the *budget sensitivity*: at 2000 FEs, performance differences between algorithms are less pronounced than at the full CEC2017 budget ($10,000 \times D$). This is evidenced by:

- The non-significant Friedman p -value ($p = 0.231$).
- The high proportion of ties in Wilcoxon tests ($48/100 = 48\%$).
- GA outperforming DE, which reverses at full budgets in the literature.

This finding has a practical implication for real-world deployment: when function evaluations are expensive (e.g., physical simulations, drug assays), ABC and GA are preferable algorithms; when cheap evaluations are available, DE’s trajectory-based approach becomes more competitive.

8 Limitations and Future Work

8.1 Current Limitations

1. **Reduced evaluation budget:** The $200 \times D = 2000$ FEs budget is $50\times$ smaller than the CEC2017 standard. Results may change significantly at full budget.
2. **Single dimension:** Only $D = 10$ was tested. Scalability analysis across $D \in \{10, 30, 50\}$ is needed for comprehensive conclusions.

3. **Fixed hyperparameters:** Algorithm parameters were set to literature-recommended defaults. Parameter tuning (e.g., grid search on w , c_1 , c_2 for PSO) could change the relative ranking.
4. **Subset of functions:** 10 out of 29 CEC2017 functions were used. The remaining composition functions are typically the most challenging.
5. **No adaptive variants:** State-of-the-art methods like LSHADE, JADE, or adaptive PSO were not included, which are known to outperform the classical variants used here.

8.2 Future Work

- Scale experiments to full CEC2017 protocol: $D \in \{10, 30, 50\}$, 51 runs, $10,000 \times D$ FEs, all 29 functions.
- Implement adaptive variants: LSHADE (DE-based), APSO, and MABC to test whether adaptive mechanisms close the budget-sensitivity gap.
- Conduct a *parameter sensitivity analysis*: vary w , F , CR , colony size by $\pm 20\%$ and measure rank stability.
- Apply the *Nemenyi post-hoc test* [demsar2006statistical] after the Friedman test to determine which specific pairs are statistically significantly different at full budget.
- Extend the framework to **multi-objective optimization** (e.g., NSGA-II, MOEA/D) and test on ZDT and DTLZ suites.
- Integrate the optimization framework as a **hyperparameter tuner** for a real machine learning model, closing the loop between theoretical benchmarks and practical deployment.

9 Conclusion

This project presented a systematic, reproducible, and statistically grounded comparative study of five metaheuristic algorithms — PSO, ABC, GA, DE, and WOA — on ten benchmark functions from the IEEE CEC2017 suite. The entire study was implemented from scratch in Python, encompassing algorithm development, a parallelized experiment runner, automated statistical analysis, and publication-quality visualization.

Key quantified findings:

- **1,500 total optimization runs** were conducted ($5 \text{ algorithms} \times 10 \text{ functions} \times 1 \text{ dimension} \times 30 \text{ runs}$) and completed in **3 minutes 7 seconds** using parallel CPU execution.

- **ABC ranked first** (Friedman average rank = 2.2) and dominated in pairwise comparisons, winning against DE on 6/10 functions and WOA on 5/10 functions (Wilcoxon, $\alpha = 0.05$).
- **GA ranked second** (average rank = 2.7) and outperformed DE (5 Wilcoxon wins vs. 3), a reversal of the expected result that is explained by GA’s elitism advantage at constrained evaluation budgets.
- **Friedman $\chi^2 = 5.60$, $p = 0.231$:** differences are directionally consistent but do not reach statistical significance at the current 2000 FEs budget, underscoring the need for full-protocol experiments.
- **WOA ranked last** (average rank = 3.6), indicating that its spiral update mechanism is insufficiently explorative at this budget and dimension.

Broader implications: The No-Free-Lunch theorem predicts that no algorithm is universally superior, and our results support this: ABC dominates on unimodal and multimodal functions but is nearly tied with GA on hybrid categories. This confirms that algorithm selection must be guided by the problem landscape rather than a single global ranking.

The modular Python codebase developed in this project is directly extensible: adding a new algorithm requires only implementing the `optimize() → (best_fit, best_pos, history)` interface; the runner, analysis, and plotting modules require no modification. This constitutes a reusable research framework suitable for future work in continuous optimization, feature selection (Project 2), and beyond.

References

- [1] Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 – International Conference on Neural Networks*, 4, 1942–1948.
- [2] Karaboga, D. (2005). An idea based on honey bee swarm for numerical optimization. *Technical Report TR06*, Computer Engineering Department, Engineering Faculty, Erciyes University.
- [3] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- [4] Storn, R., & Price, K. (1997). Differential Evolution – A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341–359.
- [5] Mirjalili, S., & Lewis, A. (2016). The Whale Optimization Algorithm. *Advances in Engineering Software*, 95, 51–67.
- [6] Wu, G., Mallipeddi, R., & Suganthan, P. N. (2017). Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective bound-constrained real-parameter numerical optimization. *Technical Report*, Nanyang Technological University.
- [7] Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67–82.
- [8] Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83.
- [9] Friedman, M. (1940). A comparison of alternative tests of significance for the problem of m rankings. *The Annals of Mathematical Statistics*, 11(1), 86–92.
- [10] Awad, N. H., Ali, M. Z., Liang, J. J., Qu, B. Y., & Suganthan, P. N. (2017). Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective real-parameter numerical optimization. *Technical Report*, NTU, Singapore.
- [11] Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.

A Project File Structure

Listing 1. Project directory tree

```

1 Project1/
2 |-- algorithms/
3 |   |-- __init__.py      # ALGORITHMS dict (PSO/ABC/GA/DE/WOA)
4 |   |-- pso.py           # Particle Swarm Optimization
5 |   |-- abc.py           # Artificial Bee Colony
6 |   |-- ga.py            # Genetic Algorithm (SBX + poly mutation)
7 |   |-- de.py            # Differential Evolution (DE/rand/1/bin)
8 |   |-- woa.py           # Whale Optimization Algorithm
9 |-- experiments/
10 |   |-- run_all.py       # Parallel experiment runner (
        ProcessPoolExecutor)
11 |-- analysis/
12 |   |-- statistical_tests.py # Wilcoxon + Friedman + summary tables
13 |   |-- plots.py         # All visualization functions
14 |-- results/
15 |   |-- all_algo_results.csv # Master results (1500 rows)
16 |   |-- all_histories.npy   # Convergence histories
17 |   |-- analysis_output/    # CSV tables from statistical_tests.py
18 |   |-- plots/              # PNG files from plots.py
19 |---- report.pdf           # This document

```

B How to Reproduce Results

Listing 2. Step-by-step reproduction commands

```

1 # 1. Create and activate virtual environment
2 python -m venv venv
3 venv\Scripts\activate      # Windows PowerShell
4
5 # 2. Install dependencies
6 pip install numpy scipy pandas matplotlib seaborn tqdm opfunu
7
8 # 3. Run all experiments (parallel, ~3-4 minutes)
9 python experiments/run_all.py
10
11 # 4. Run statistical analysis
12 python analysis/statistical_tests.py
13
14 # 5. Generate all plots
15 python analysis/plots.py
16
17 # Results will be in:
18 #   results/all_algo_results.csv

```

```

19 # results/analysis_output/*.csv
20 # results/plots/*.png

```

C Algorithm Hyperparameter Table

Table 8. Algorithm Hyperparameters Used in Experiments

Algorithm	Parameter	Value
PSO	Inertia weight w	0.7
	Cognitive c_1	1.5
	Social c_2	1.5
ABC	Colony size	20 (10 employed + 10 onlooker)
	Limit	$n_{\text{emp}} \times D = 100$
GA	Crossover prob p_c	0.9
	Mutation prob p_m	$1/D = 0.1$
	SBX index η_c	20
	Polynomial mut. η_m	20
DE	Scale factor F	0.8
	Crossover rate CR	0.9
WOA	Spiral constant b	1.0
	a decay	Linear: $2 \rightarrow 0$
Population / colony size (all)		20
Max function evaluations		$200 \times D = 2000$
Independent runs		30